Packages, Modules & Generics Cheatsheet

Lessons: 16 — Packages & Modules · 17 — Generics

Examples: 17

Covers: package design, `internal/`, module versioning, type parameters, constraints

Legend: `[*]` = real Go API/feature that the lessons have not covered yet

PACKAGES

```
package store                 one package per directory, named after the dir
import "example.com/proj/store"    module path + directory path
store.New(...)                callers always say the package name
Exported / unexported         capital letter = visible outside the package
package main                  the only package that produces a binary
package foo_test          [*] external test package: tests the public API only
func init()                   runs after package vars, before main; avoid it
(no cyclic imports, ever — Go refuses to compile them)
```

PACKAGE NAMING

```
short, lowercase, no underscores    store, user, httputil
no plurals                    package store, not stores
no stutter                    store.New, not store.NewStore
no util / common / helpers    name packages after what they PROVIDE
user.Service                  reads well from the caller's side
user.UserService              stutter — drop the prefix
one purpose per package       if you can't name it, it does too much
```

PROJECT LAYOUT

```
cmd/api/main.go               one directory per binary
internal/                     importable ONLY inside this module — enforced
internal/user/                a domain package
pkg/                      [*] "safe for others to import" — often unnecessary
api/ or proto/            [*] schema/contract files
migrations/                   SQL migrations
testdata/                     ignored by the toolchain; test fixtures live here
vendor/                   [*] committed dependencies (go mod vendor)
(start flat; add directories when a file gets hard to name)
```

MODULES

```
module example.com/proj       the import path prefix for every package
go 1.24                       the language version this module targets
require example.com/lib v1.2.3   a direct dependency
require example.com/x v1.0.0 // indirect   pulled in by a dependency
replace example.com/foo => ../foo    local development against a fork
exclude example.com/x v1.4.0 [*] refuse one bad version
go.sum                        hashes of every module version ever used here
```

VERSIONING

```
vMAJOR.MINOR.PATCH           semantic versioning, with a leading v
v0.x.y                       no compatibility promise
v1.x.y                       breaking changes are not allowed
/v2 in the module path       major versions ≥ 2 change the IMPORT PATH
github.com/foo/bar/v2        so v1 and v2 can coexist in one build
minimal version selection    Go picks the LOWEST version that satisfies all
git tag v1.2.3               publishing is just a tag
go get pkg@v1.2.3            an exact version
go get pkg@latest            the newest release
go list -m all               the resolved build list
```

GENERICS: syntax

```
func Map[T, U any](s []T, f func(T) U) []U     type parameters in brackets
Map(nums, double)            usually INFERRED from the arguments
Map[int, string](nums, f)    explicit instantiation when inference fails
type Stack[T any] struct { items []T }     a generic type
var s Stack[int]             instantiate with a concrete type
func (s *Stack[T]) Push(v T) methods repeat the parameter, add none of their own
var zero T                   the zero value of an unknown type
type Pair[K comparable, V any] struct { Key K; Val V }   several parameters
```

CONSTRAINTS

```
any                          no constraint — you can only pass it around
comparable                   supports == and != (map keys, slices.Contains)
cmp.Ordered              [*] Go 1.21+: < <= > >= (the stdlib home for it)
constraints.Ordered      [*] the older golang.org/x/exp version
interface { ~int | ~float64 }   a type SET: either underlying type
~int                         the ~ means "any type whose underlying type is int"
                             so `type Celsius float64` satisfies ~float64
interface { Method() }       an ordinary interface is also a constraint
interface { ~string; Len() int }   combine a type set with a method
```

GENERIC STDLIB (Go 1.21+) [*]

```
slices.Sort / SortFunc / Contains / Index / Max / Min / Clone / Equal
maps.Keys / Values / Clone / Equal / DeleteFunc / Copy
cmp.Compare(a, b)            -> -1, 0, +1 for ordered types
cmp.Or(a, b, ...)            the first non-zero value — multi-key sorting
sync.OnceValue[T]            a typed lazy value
atomic.Pointer[T]            a typed atomic pointer
iter.Seq[T] / iter.Seq2[K,V] the range-over-func iterator types (Go 1.23+)
```

GENERIC BUILDING BLOCKS (what you actually write)

```
transform          Map[T,U](s []T, f func(T) U) []U
                   MapValues[K,V,W](m map[K]V, f func(V) W) map[K]W
select             Filter[T](s []T, keep func(T) bool) []T
                   Partition[T](s []T, pred func(T) bool) (yes, no []T)
fold               Reduce[T,A](s []T, init A, f func(A, T) A) A
                   Frequency[T comparable](s []T) map[T]int
                   GroupBy[T,K comparable](s []T, key func(T) K) map[K][]T
                   Associate[T,K comparable](s []T, key func(T) K) map[K]T
reshape            Chunk[T](s []T, n int) [][]T
                   Flatten[T](ss [][]T) []T
                   Zip[A,B](a []A, b []B) []Pair[A,B]
                   Unique[T comparable](s []T) []T
compare            Equal / IndexOf / Contains — need `comparable`
                   Min / Max / Clamp / SortBy — need `cmp.Ordered`
containers         Stack[T] / Queue[T] / Set[T] / Pair[K,V] / LinkedList[T]
                   Tree[T cmp.Ordered] / LRU[K comparable, V]
                   — a generic container is where generics earn their keep
optionals          Optional[T] struct{ v T; ok bool }
                   Result[T] struct{ v T; err error }
                   (Go prefers (T, bool) and (T, error) — use these sparingly)
functional         Memoize[K comparable, V](f func(K) V) func(K) V
                   Compose[A,B,C](f func(A) B, g func(B) C) func(A) C
                   — clever, and rarely clearer than writing the call out
typed registries   EventBus[T] / Repository[T, ID] — one implementation,
                   many entity types, no `any` and no type assertions
(the collection ops above are covered in depth on the 54–55 sheet)
```

WHEN NOT TO USE GENERICS

```
one concrete type            just write it for that type
an interface already fits    io.Writer beats [T Writer]
you need reflection anyway   generics don't give you field access
the constraint has 1 member  a type parameter with one option is a type
readability drops            three type parameters is usually a design smell
(rule: write it twice concretely, THEN generalize if it still hurts)
```

TRAPS & MEMORIZE

```
cyclic imports               a compile error; extract a third package
internal/ is enforced        by the compiler, not convention
package name ≠ directory     legal but confusing; keep them the same
init() ordering              vars, then init(), across files alphabetically
v2 without the path change   `go get` silently keeps giving people v1
go.sum conflicts             run go mod tidy, never hand-edit
methods can't add type params  func (s Stack[T]) Map[U any]() is illegal
comparable ≠ Ordered         comparable is ==, Ordered is <
generic code isn't faster    it can be slower than an interface for one type
type inference limits        return-only type parameters must be explicit
```